EXP:24

PROGRAM CODE:

```
// Exp = 24
// NAME: M DHANESH
// REG NO: 192512348
// ECA 1408: EMBEDDED SYSTEM

#include <systemc.h>
#include <iostream>

SC_MODULE(ClockExample) {
   sc_in<bool> clk;

   void process() {
      while (true) {
         wait();
         std::cout << "Time: " << sc_time_stamp()
                  << " - Clock Triggered" << std::endl;
      }
   }

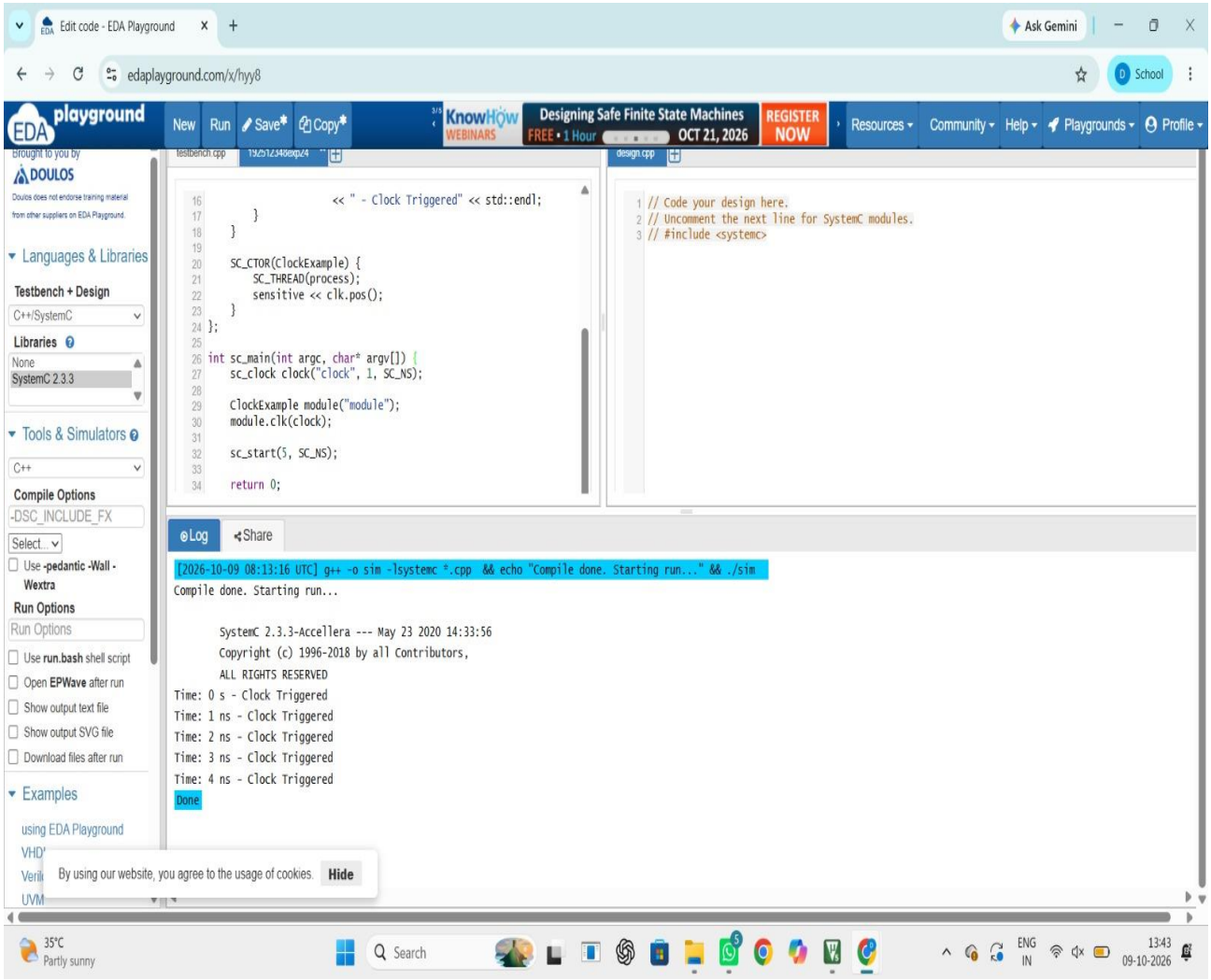
   SC_CTOR(ClockExample) {
      SC_THREAD(process);
      sensitive << clk.pos();
   }
};
int sc_main(int argc, char* argv[]) {
   sc_clock clock("clock", 1, SC_NS);
```

```
    ClockExample module("module");
    module.clk(clock);

    sc_start(5, SC_NS);

    return 0;
}
```

RESULT:



EXP:25

PROGRAM CODE:

```
#include <systemc.h>
#include <iostream>

// Producer Module
SC_MODULE(Producer) {
   sc_fifo<int> out;

   void produce() {
      for (int i = 1; i <= 5; i++) {
         std::cout << "Produced: " << i << std::endl;
         out.write(i);
         wait(1, SC_SEC);
      }
   }

   SC_CTOR(Producer) : out("out", 5) {
      SC_THREAD(produce);
   }
};

// Consumer Module
SC_MODULE(Consumer) {
   sc_fifo<int> in;

   void consume() {
      int data;
      for (int i = 1; i <= 5; i++) {
```

```
            data = in.read();
            std::cout << "Consumed: " << data << std::endl;
        }
    }

    SC_CTOR(Consumer) : in("in", 5) {
        SC_THREAD(consume);
    }
};

// Top Module
SC_MODULE(Top) {
    Producer *p;
    Consumer *c;
    sc_fifo<int> channel;

    SC_CTOR(Top) : channel("channel", 5) {
        p = new Producer("producer");
        c = new Consumer("consumer");

        p->out(channel);
        c->in(channel);
    }

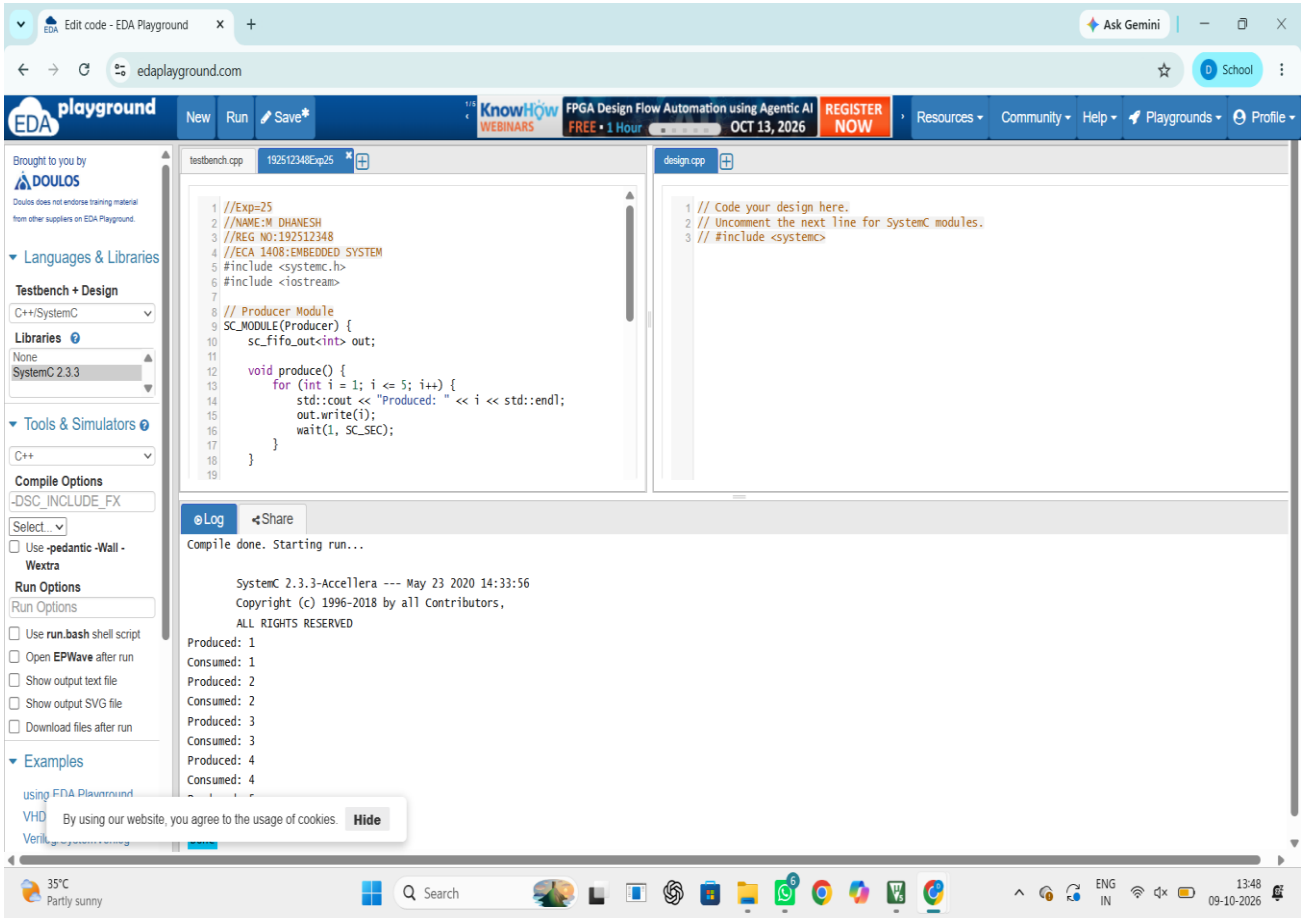
    ~Top() {
        delete p;
        delete c;
    }
};

int sc_main(int argc, char* argv[]) {
    Top top("top");
```

```
    sc_start(6, SC_SEC);

    return 0;
}
```

OUTPUT:



EXP:26

PROGRAM CODE:

```
#include <systemc>
#include <tlm>
#include <tlm_utils/simple_initiator_socket.h>
#include <tlm_utils/simple_target_socket.h>
#include <iostream>

using namespace sc_core;
using namespace tlm;
using namespace std;

// Target Module
struct Target : sc_module {
  tlm_utils::simple_target_socket<Target> socket;

  SC_CTOR(Target) : socket("socket") {
    socket.register_b_transport(this, &Target::b_transport);
  }

  void b_transport(tlm_generic_payload& trans, sc_time& delay) {
    int* data = reinterpret_cast<int*>(trans.get_data_ptr());

    cout << "Target received: " << *data << endl;
    delay += sc_time(10, SC_NS);
  }
};
```

```
// Initiator Module
struct Initiator : sc_module {
  tlm_utils::simple_initiator_socket<Initiator> socket;

  SC_CTOR(Initiator) : socket("socket") {
    SC_THREAD(thread_process);
  }

  void thread_process() {
    tlm_generic_payload trans;
    sc_time delay = SC_ZERO_TIME;
    int data = 555;

    trans.set_command(TLM_WRITE_COMMAND);
    trans.set_address(0);
    trans.set_data_ptr(
      reinterpret_cast<unsigned char*>(&data));
    trans.set_data_length(sizeof(data));
    trans.set_streaming_width(sizeof(data));
    trans.set_byte_enable_ptr(nullptr);
    trans.set_dmi_allowed(false);
    trans.set_response_status(TLM_INCOMPLETE_RESPONSE);

    cout << "Initiator sending: " << data << endl;

    socket->b_transport(trans, delay);

    wait(delay);
  }
};
```

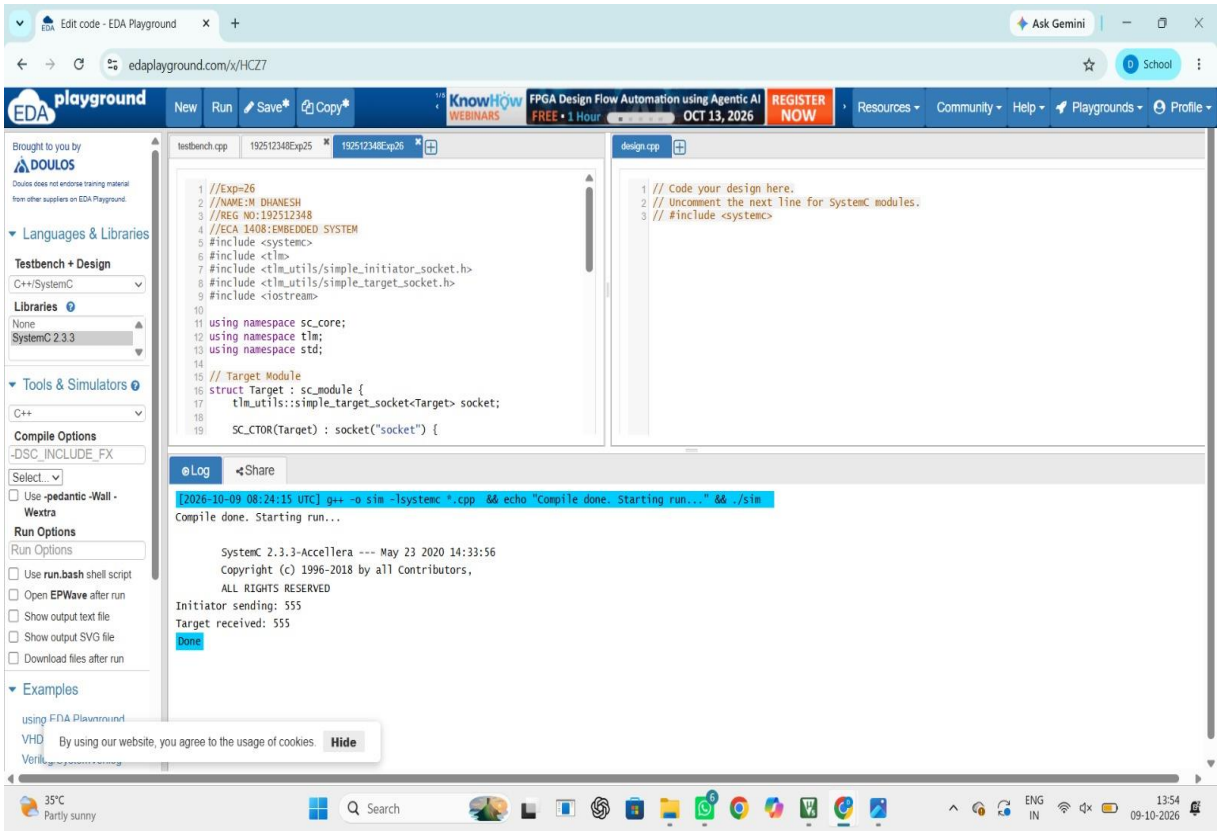
```
int sc_main(int argc, char* argv[]) {
    Initiator initiator("initiator");
    Target target("target");

    initiator.socket.bind(target.socket);

    sc_start();

    return 0;
}
```

OUTPUT:



```
```

EXP:27

PROGRAM CODE:

```
#include <systemc.h>

SC_MODULE(binary_semaphore) {
   int sem;
   sc_event ev;

   void task1() {
      while (true) {
         wait(1, SC_SEC);

         if (sem == 1) {
            sem = 0;
            cout << "Task1 Entered Critical Section at "
                 << sc_time_stamp() << endl;

            wait(2, SC_SEC);

            sem = 1;
            ev.notify();
         } else {
            wait(ev);
         }
      }
   }

   void task2() {
      while (true) {
```

```
        wait(2, SC_SEC);

        if (sem == 1) {
            sem = 0;
            cout << "Task2 Entered Critical Section at "
                 << sc_time_stamp() << endl;

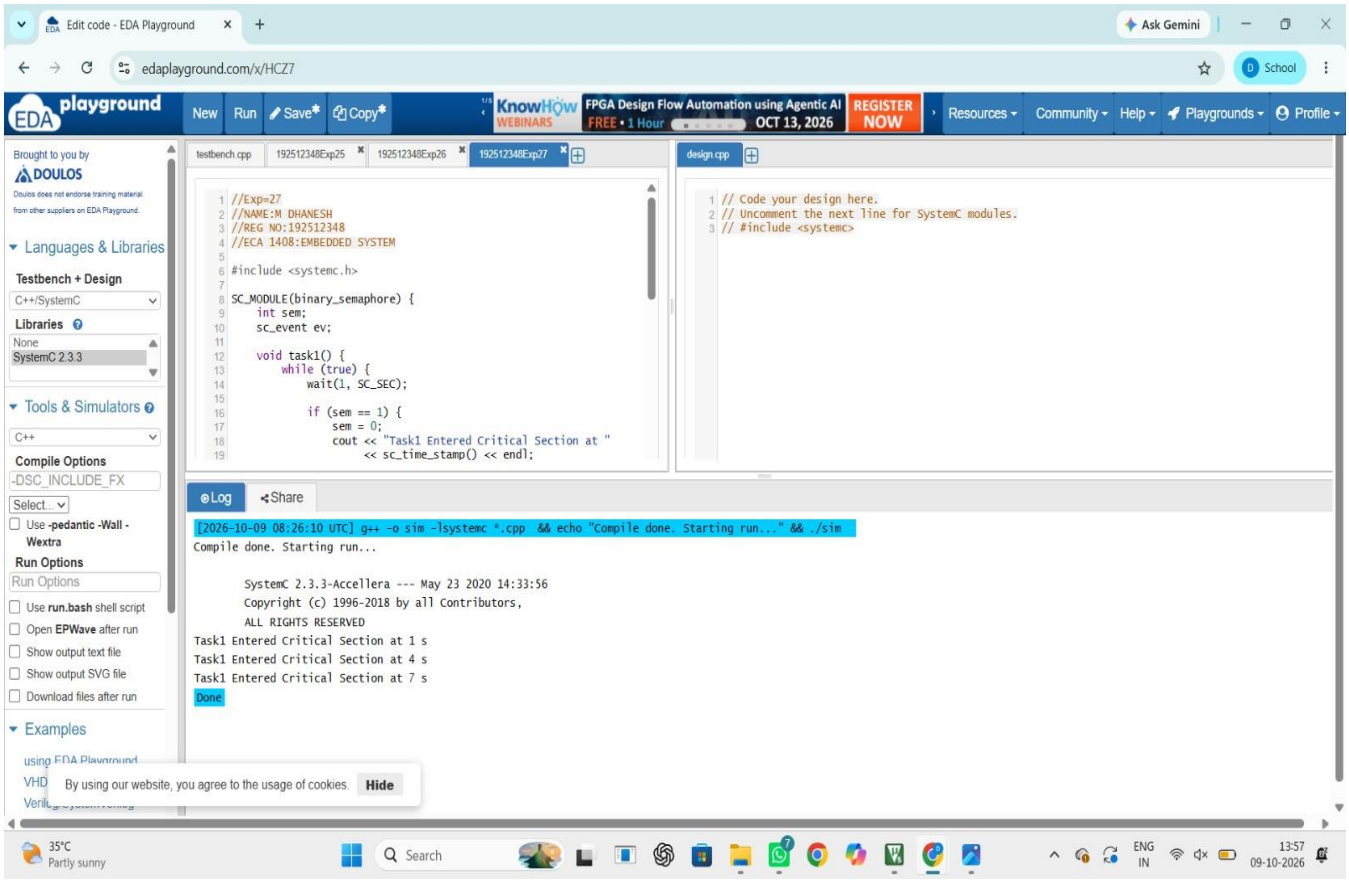
            wait(1, SC_SEC);

            sem = 1;
            ev.notify();
        } else {
            wait(ev);
        }
    }
  }

  SC_CTOR(binary_semaphore) {
    sem = 1;
    SC_THREAD(task1);
    SC_THREAD(task2);
  }
};

int sc_main(int argc, char* argv[]) {
  binary_semaphore obj("Binary_Semaphore");
  sc_start(10, SC_SEC);
  return 0;
}
```

OUTPUT:


Edit code - EDA Playground
edaplayground.com/x/HCZ7
playground
New
Run
Save
Copy
FPGA Design Flow Automation using Agentic AI
FREE • 1 Hour
OCT 13, 2026
REGISTER NOW
Resources
Community
Help
Playgrounds
Profile
testbench.cpp
192512348Exp25
192512348Exp26
192512348Exp27
design.cpp
//Exp=27
//NAME:M DHANESH
//REG NO:192512348
//ECA 1408:EMBEDDED SYSTEM
#include <systemc.h>
SC_MODULE(binary_semaphore) {
int sem;
sc_event ev;
void task1() {
while (true) {
wait(1, SC_SEC);
if (sem == 1) {
sem = 0;
cout << "Task1 Entered Critical Section at "
<< sc_time_stamp() << endl;
// Code your design here.
// Uncomment the next line for SystemC modules.
// #include <systemc>
Log
Share
[2026-10-09 08:26:10 UTC] g++ -o sim -lsystemc *.cpp && echo "Compile done. Starting run..." && ./sim
Compile done. Starting run...
SystemC 2.3.3-Accellera --- May 23 2020 14:33:56
Copyright (c) 1996-2018 by all Contributors,
ALL RIGHTS RESERVED
Task1 Entered Critical Section at 1 s
Task1 Entered Critical Section at 4 s
Task1 Entered Critical Section at 7 s
Done
Languages & Libraries
Tools & Simulators
Examples
By using our website, you agree to the usage of cookies.
Hide
35°C
Partly sunny
Search
ENG
IN
13:57
09-10-2026

EXP:28

PROGRAM CODE:

```
#include <systemc.h>

SC_MODULE(counting_semaphore) {
  int sem;

  void task1() {
    while (true) {
      wait(1, SC_SEC);

      if (sem > 0) {
        sem--;
        cout << "Task Accessing Resource at "
          << sc_time_stamp()
          << " Remaining: " << sem << endl;
      }
    }
  }

  void task2() {
    while (true) {
      wait(1, SC_SEC);

      if (sem > 0) {
        sem--;
        cout << "Task Accessing Resource at "
          << sc_time_stamp()
          << " Remaining: " << sem << endl;
      }
```

```
        }
    }

    void task3() {
        while (true) {
            wait(1, SC_SEC);

            if (sem > 0) {
                sem--;
                cout << "Task Accessing Resource at "
                    << sc_time_stamp()
                    << " Remaining: " << sem << endl;
            }
        }
    }

    SC_CTOR(counting_semaphore) {
        sem = 2;

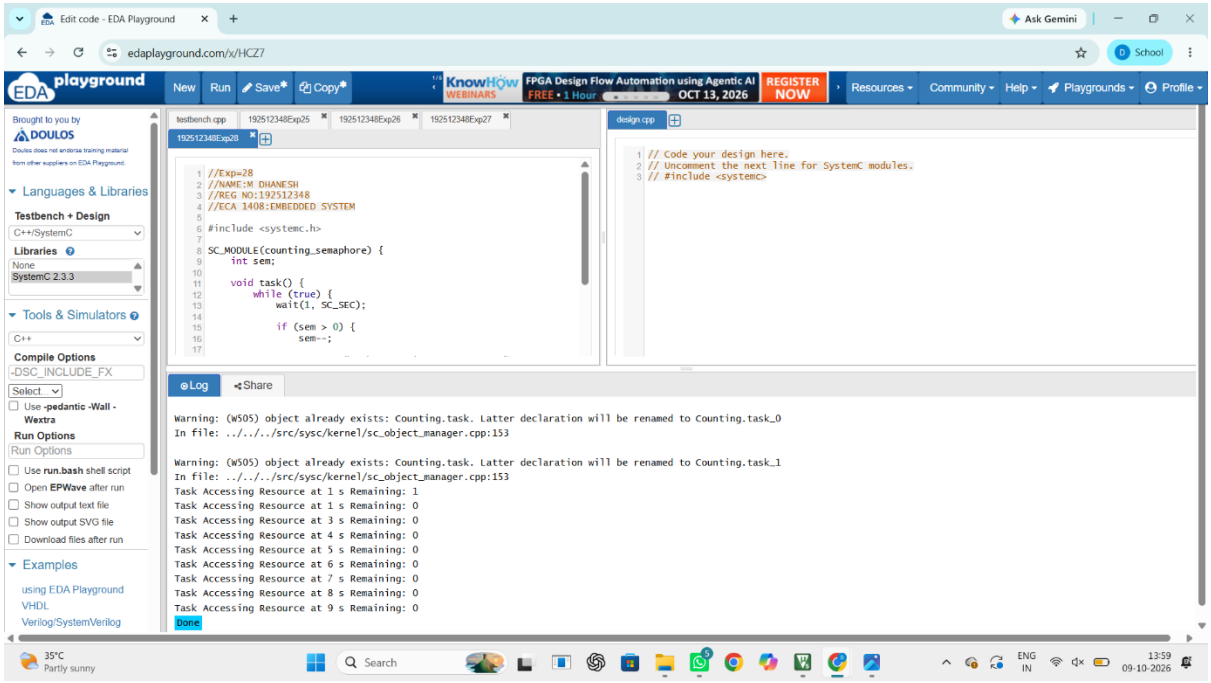
        SC_THREAD(task1);
        SC_THREAD(task2);
        SC_THREAD(task3);
    }
};

int sc_main(int argc, char* argv[]) {
    counting_semaphore cs("cs");

    sc_start(10, SC_SEC);

    return 0;
}
```

OUTPUT:


Edit code - EDA Playground
edaplayground.com/x/HCZ7
playground
New
Run
Save*
Copy*
FPGA Design Flow Automation using Agentic AI
OCT 13, 2026
REGISTER NOW
Resources
Community
Help
Playgrounds
Profile
Languages & Libraries
Testbench + Design
C++/SystemC
Libraries
SystemC 2.3.3
Tools & Simulators
Compile Options
Run Options
Examples
testbench.cpp
192512348Exp25
192512348Exp26
192512348Exp27
192512348Exp28
design.cpp
//Exp=28
//NAME:M DHANESH
//REG NO:192512348
//ECA 1408:EMBEDDED SYSTEM
#include <systemc.h>
SC_MODULE(counting_semaphore) {
int sem;
void task() {
while (true) {
wait(1, SC_SEC);
if (sem > 0) {
sem--;
// Code your design here.
// Uncomment the next line for SystemC modules.
// #include <systemc>
Log
Share
Warning: (WSOS) object already exists: Counting.task. Latter declaration will be renamed to Counting.task_0
In file: ../../../src/sysc/kernel/sc_object_manager.cpp:153
Warning: (WSOS) object already exists: Counting.task. Latter declaration will be renamed to Counting.task_1
In file: ../../../src/sysc/kernel/sc_object_manager.cpp:153
Task Accessing Resource at 1 s Remaining: 1
Task Accessing Resource at 1 s Remaining: 0
Task Accessing Resource at 3 s Remaining: 0
Task Accessing Resource at 4 s Remaining: 0
Task Accessing Resource at 5 s Remaining: 0
Task Accessing Resource at 6 s Remaining: 0
Task Accessing Resource at 7 s Remaining: 0
Task Accessing Resource at 8 s Remaining: 0
Task Accessing Resource at 9 s Remaining: 0
Done
35°C
Partly sunny
Search
ENG
IN
13:59
09-10-2026

EXP:29

PROGRAM CODE:

```
#include <systemc.h>

SC_MODULE(priority_interrupt) {
   sc_event low_int, high_int;

   void low_ISR() {
      while (true) {
         wait(low_int);
         cout << "Low Priority Interrupt at "
              << sc_time_stamp() << endl;
         wait(3, SC_SEC);
      }
   }

   void high_ISR() {
      while (true) {
         wait(high_int);
         cout << "High Priority Interrupt at "
              << sc_time_stamp() << endl;
         wait(1, SC_SEC);
      }
   }

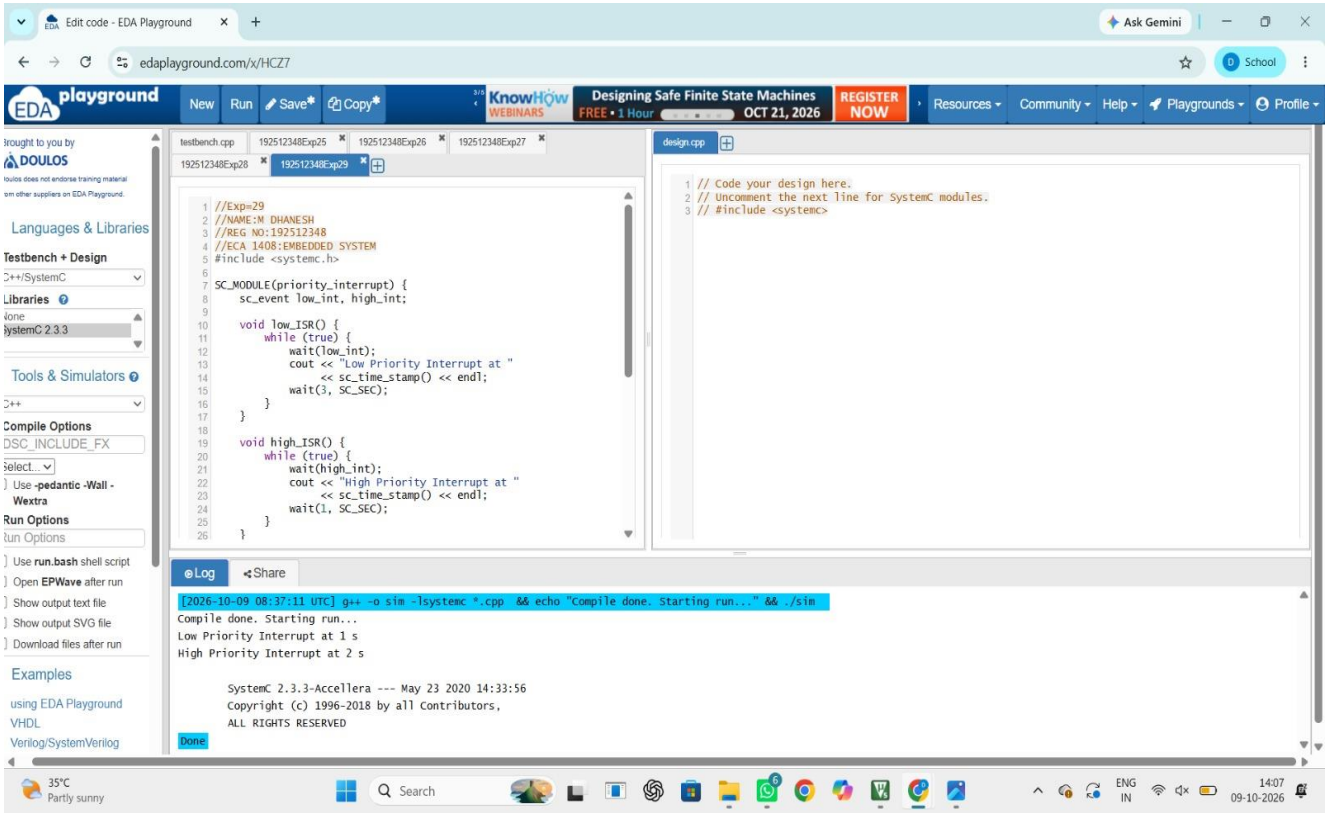
   void generator() {
      wait(1, SC_SEC);
      low_int.notify();
```

```
        wait(1, SC_SEC);
        high_int.notify();
    }

    SC_CTOR(priority_interrupt) {
        SC_THREAD(low_ISR);
        SC_THREAD(high_ISR);
        SC_THREAD(generator);
    }
};

int sc_main(int argc, char* argv[]) {
    priority_interrupt obj("Priority_INT");
    sc_start(10, SC_SEC);
    return 0;
}
```

OUTPUT:



EXP:30

PROGRAM CODE:

```
#include <systemc.h>

SC_MODULE(rtos_scheduler) {

  void task1() {
    while (true) {
      cout << "Task1 Running at "
          << sc_time_stamp() << endl;
      wait(1, SC_SEC);
    }
  }

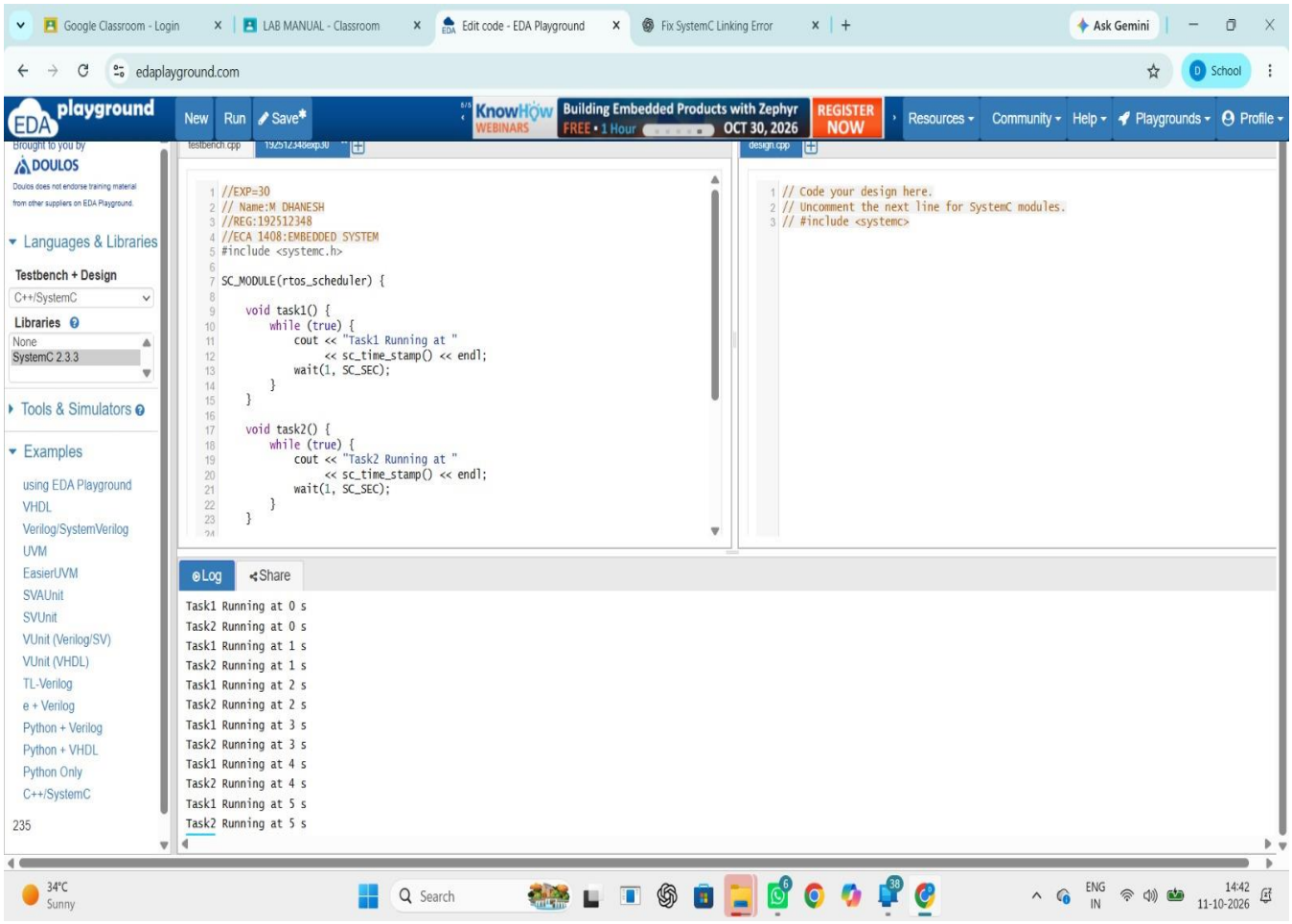
  void task2() {
    while (true) {
      cout << "Task2 Running at "
          << sc_time_stamp() << endl;
      wait(1, SC_SEC);
    }
  }

  SC_CTOR(rtos_scheduler) {
    SC_THREAD(task1);
    SC_THREAD(task2);
  }
};

int sc_main(int argc, char* argv[]) {
```

```
    rtos_scheduler obj("Scheduler");
    sc_start(6, SC_SEC);
    return 0;
}
```

OUTPUT:



Generated by PDFengine · 2026-11-01T16:51:13Z · Verify at pdfen.com/verify/7H724F29JX